

M5 Forecasting Accuracy

Forecasting 28 Days of Walmart Unit Sales with Machine Learning

Chingfen Hung
Davey Johnson
Ko-Jung Hsu
Evelyn Lai
Tzu-Yu Chen

3,049

Unique Products Tracked

28 Days

Forecast Horizon

0.59212

Best Kaggle Score

Business Context

THE CHALLENGE

Forecast 28 days of unit sales for Walmart product-store series across 10 stores in 3 U.S. states, using historical sales, calendar events, SNAP days, and price information.

WHY IT MATTERS



Inventory Efficiency

Reduce overstock and stockout risk through better demand visibility



Proactive Planning

Help teams plan four weeks ahead instead of reacting to shortages



Cost Reduction

Lower waste, holding costs, and unnecessary replenishment



Promotion & Event Readiness

Capture demand shifts from price changes, SNAP days, holidays, and local events

Data Cleaning & Preparation

INPUT → OUTPUT
3 CSVs → 1 Clean File

PIPELINE STEPS:

1

Parse types & fill missing values

Dates parsed to datetime, 1,807 event NaNs filled

2

Downcast columns

Sales int64 → int16, prices float64 → float32

3

Detect leading zeros

77% of series have pre-shelf zeros flagged

4

Melt wide → long & merge

Combined features into one row-per-item-day table

5

Forward-fill price gaps

Items with late price records filled using first known price

6

Export clean parquet

Single file for all downstream modeling notebooks

KEY NUMBERS:

16M+

Rows After
Melt

3,049

Unique Products
Tracked

60%

Of Products Had
No Daily Sales

ISSUES FOUND & RESOLVED



1,807 event NaNs — not missing data, just "no event"; filled with no_event variable



77% leading zeros — items not yet on shelf; masked with is_active flag



Price gaps after melt — items with late price records; forward/back-filled per unique product

Baseline Model Building - LightGBM

BASELINE RMSE

2.1619

WHAT IS LightGBM?



LightGBM is a gradient boosting model that designed to handle large-scale tabular data fast, and natively supports categorical features without manual encoding.



Splits data into
decision trees



Corrects errors
step by step



Outputs sales
prediction per row

FEATURE LOGIC

Categorical Handled natively by LightGBM

Calendar Captures weekly cycles, seasonality,
and government benefit days

Price / Promo Models consumer price sensitivity and
promotional lift

Lag & Rolling Encodes recent demand momentum

MODEL SETUP

Objective

Tweedie

Suited for sparse count data with many zeros

Sample

2,000 products

× 500 days

Features

14 total

Lag, rolling means, calendar, price signals

Metric

RMSE

Root mean squared error; lower = better

Max Rounds

2,000

Early stopping @ 100 rounds

Learn Rate

0.05

Found via Bayesian optimization

Num Leaves

127

Controls tree complexity

METHODOLOGY

Iteratively add feature groups to the baseline model, then evaluate each on a held-out validation window using RMSE, with all lag and rolling features computed shift-first to prevent data leakage.

KEY RESULT

Three extra features (snap_flag, is_event, lag_56) beat complex combinations of other features.

FINAL FEATURE TABLE

CATEGORICAL

item_id Item identifier

dept_id Department ID

cat_id Category ID

store_id Store identifier

state_id State identifier

CALENDAR

dow Day of week

month Month of year

snap_flag SNAP benefit day

is_event Holiday / event

PRICE / PROMO

sell_price Item selling price

price_change Price change vs.
prev day

is_promo Promotion flag

LAG & ROLLING

lag_7 7-day lagged
sales

lag_28 28-day lagged
sales

lag_56 56-day lagged
sales

rmean_7 7-day rolling
mean

rmean_28 28-day rolling
mean

● New features added on top of baseline

Fine-Tuning: Finding the Best Model Settings

LightGBM RMSE

2.0026

Approach: 2-Phase Search Sample: 1,000 products × 500 days

Phase	Method	Trials
Phase 1	Manual Grid Search	12 sets
Phase 2	Automated Smart Search (Bayesian Optimization)	40 trials

Best Parameters Found

Parameter	Value	Meaning
Learning Rate	0.042	How fast the model learns
Tree Complexity	91 leaves	Level of detail captured
Demand Pattern	1.28	Tuned for sparse sales data
Min Data in Leaf	85	Lower = better

Model Exploration: Ridge Regression

RIDGE REGRESSION RMSE

2.1180

WHAT IS RIDGE REGRESSION?



Ridge regression is a linear model which incorporates regularization, it finds the best straight-line fit while making sure the model can still predict well with new data.



Fits a line across a graph



Shrinks noise within data



Predicts next-day sales per product

Ridge Model Setup

- Alpha = 0.5 (regularization strength)
- StandardScaler applied to all features
- Same engineered features as team models
- Same validation window for comparison

MODEL COMPARISON

Lower RMSE = better

Ridge Regression

Linear baseline

2.1180

Random Forest

Tree ensemble

2.0802

GRU

Deep learning

2.2583

Takeaways

Strength

- Fast to train, fully interpretable

Weaknesses

- Assumes linear relationships
- Produces negative predictions

Model Exploration: Random Forest

RANDOM FOREST RMSE

2.0802

MODEL COMPARISON

Lower RMSE = better

Ridge Regression

Linear baseline

2.1180

Random Forest

Tree ensemble

2.0802

GRU

Deep learning

2.2583

Takeaways

Strength

- Robust to outliers and skewed data

Weaknesses

- Slower to train than Ridge or GRU
- Cannot predict beyond the training range

WHAT IS RANDOM FOREST?

Random Forest builds hundreds of decision trees on random subsets of data, then averages their predictions, can be seen as polling many experts instead of trusting one.



Grows 300 decision trees



Each tree sees random features



Averages votes for final prediction

Random Forest Model Setup

- 300 trees, max depth 20
- Min 20 samples per leaf
- Same engineered features as team models
- Same validation window for comparison

Model Exploration: GRU

GRU RMSE
2.2583

WHAT IS GRU?



A Gated Recurrent Unit (GRU) is a neural network that reads historical sales as a sequence — much like how a human analyst reviews past weeks before making a forecast.



Reads 28-day
history window



Learns demand
patterns over time



Predicts next-day
sales per product

GRU Model Setup

- 28-day input window
- Same engineered features as team models
- Same validation window for comparison

MODEL COMPARISON

Lower RMSE = better

Ridge Regression

Linear baseline

2.1180

Random Forest

Tree ensemble

2.0802

GRU

Deep learning

2.2583

Takeaways

Strength

- Captures sequential demand patterns

Weaknesses

- Less effective on days with zero sales
- Performed the worst out of all models

Final Prediction: LightGBM

KAGGLE SCORE

0.59212

Recursive Forecasting:

1. Get static features (calendar, price, events)
2. Compute lag_7, lag_28, lag_56 from history
3. Compute rmean_7, rmean_28 from history
4. Predict demand for each ID
5. Clip negative predictions to 0

Append predictions to history → repeat

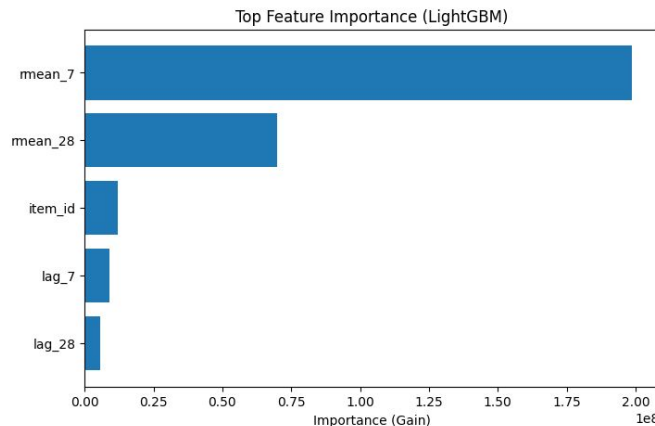
💡 Enables multi-step prediction without retraining the model

Ensemble Strategy:

Train multiple models with three different seeds

👉 Reduces variance & Improves stability

📈 ~0.001 WRMSSE improvement vs single model



The model learns both temporal dynamics and item-level behavior

Recommendations

Business Impact

- Improves visibility into future demand of 3,049 products sold across 10 stores
- Helps inventory teams plan and allocate stock earlier
- Reduces waste from over- or under-preparation

Risks & Limitations

- Sudden demand shifts (e.g. supply disruptions) may reduce accuracy
- Promotions or local events may need manual adjustment
- Forecast uncertainty increases further out in the 28-day horizon

Next Steps

- Refresh the model periodically with new sales data
- Incorporate additional business context (e.g. pricing, events) into future forecasts
- Use forecast output alongside manager judgment — not as a replacement